

Stream Engine 工作机制说明

1. Stream Engine 基本结构

1.1 iCntMap（迭代计数器）

在流式计算中（如向量加 $a[i] + b[i] \rightarrow c[i]$ ），每次执行一条 CALSTREAM（流计算指令）时，硬件必须知道：

这一条计算指令要使用 Buffer 中的第几个元素？

由于在流式计算中，操作数索引一般都会按某种规律变化，所以流计算指令的指令码本身不必携带 **各个流操作数在相应流Buffer中的索引**，各操作数的索引是指令来到流水线时动态获取的

编码时，假若把操作数索引编址在指令码中，则RF级需要先从通用寄存器中读出操作数索引，再利用该索引从流Buffer中取数，这会影响主频

例如：

- 第一次计算 → 使用 Buffer[0]
- 第二次计算 → 使用 Buffer[1]
- ……以此类推

为了实现这一点，每条流都对应一个迭代计数器，统一存放在 **iCntMap** 里

我们需要维护的逻辑是：

1. 计数器何时变化
2. 计数器按何种方式变化
3. 多发射的每条流计算指令如何根据计数器获取属于自己的操作数索引

1.2 stream Buffer（每个 stream 一个）

每个 **stream** 流 分配：

- 一个 Buffer，包含 **X** 项

- 每项是 **4B word**
- 每项配套一个 flag，称为 **消费计数**

其中：

$$X = 2 \times \text{L2CacheLine_WORD}$$

即：每个流buffer是一个 有两个 cacheline 规模的双缓冲结构，分为两个半区

1.3 readyMap

项的个数和Buffer项的个数一样，每一项为若干bit

流类型	ready > 0 (= x)	ready = 0
Load 流	数据已经填充到 FIFO，可被计算指令消费x次	空闲，可从主存读取数据填充到 FIFO
Store 流	FIFO 中有计算结果，准备写回主存	FIFO 空闲，计算单元可写入新结果

1.4 配置表

每个流（Stream）在使用前需要配置完整的访存与索引更新信息。整体上，这些配置可分为 **访存相关信息** 和 **索引更新相关信息** 两大类。

1.4.1 访存相关信息

该类配置用于描述流在主存中的数据布局以及访存方式，决定 **数据从哪里来、以什么步长取、如何复用**。

基本约定（统一抽象）

对于任意一个流，其数据访问层次结构统一抽象为：

流 (Stream) → Block → Tile → Word

- 每个 **流** 由若干个 **Block** 组成（每个Block中的数据一致）
- 每个 **Block** 由若干个 **Tile** 组成
- 每个 **Tile** 由若干个 **Word** 组成

需要配置的信息如下：

- **addrCfg** 每个流在主存中的起始地址（Base Address）。
- **outerIterMap** 每个流对应的 **Block** 数量
- **lengthMap** 每个 Block 对应的 **Tile** 数量
- **tilestride** 相邻 **Tile** 首地址之间的步长
- **stride** 相邻 **Word** 之间的地址步长
- **reuseMap** 描述每个 **Word** 被消费（使用）的次数，用于支持数据复用

通过上述配置，流引擎可以在 Block / Tile / Word 多个层次上，精确地生成访存地址并完成高效的数据获取。

1.4.2 索引更新相关信息

该类配置用于描述 **索引计数器 (icntmap)** 的更新规律，从而控制流中数据的访问顺序。

索引更新抽象模型

所有 icntmap 的更新模式，均可统一抽象为如下形式：

```
0          → limit
0          → limit      (重复 repeat 次)
...
limit+1 → 2×limit
limit+1 → 2×limit      (重复 repeat 次)
...
```

即：

- 索引在一个固定区间内递增
- 每个区间内的访问模式重复若干次
- 然后跳转到下一个区间继续执行

索引更新配置项

- **limitMap** 指定每一段索引递增区间的 **上界 (limit)**，决定单次索引扫描的范围大小。
- **repeatMap** 指定每个索引区间内的 **重复次数 (repeat)**，用于描述同一索引区间被重复访问的次数。

2. 流与存储系统交互 (Load / Store 流)

V2: 流Engine 连 L2-Cache & AXI

- 动机：
 1. 顺序流不需要Cache作为缓存，应直接向AXI发送请求
 2. 需要Cache缓存的流，需缓存的数据没必要来到L1，因此直连L2
- 与L2的连接
 1. SE与L2以流水的方式连接，SE仍然是维护四级流水，L2部分复用了部分channel2的数据通路
 2. SE的请求与L1-D的请求在L2的C2S1会有一个竞争的关系，目前出于实现难度的考虑，允许L1的请求优先通过
 3. L2给L1-D的接口中读数据是一整个l1-cacheline，而给SE的读数据是一个word，当L2 hit时，数据能够以流水的形式每cycle给SE返回一个数据
- 与AXI的连接

参考V0部分的介绍，基本没有改动
- 配置问题：通过简单的指令即可配置某个load流是向L2 还是向AXI发送请求

Store流仍然直连AXI

V1：流Engine 只连 L1-Cache（动机见“矩阵乘的流”部分）

Stream Engine不再通过AXI向主存发送访存请求，而是直接向Cache发送请求。Stream Engine内部维护一个类似访存流水线的4级流水：

访存请求发送（RF级）

在RF级发送访存请求（会阻塞访存流水线 RF级的请求）。先前通过AXI访存的粒度是流Buffer一个半区的长度，而从L1Cache访存的粒度是单字，所以需要维护一个循环计数器loadWordCnt记录向L1Cache发送了多少个访存请求，地址由基地址和计数器共同决定：

```
loadAddr = addrDyn(loadFifoIdReg) + loadWordCnt * strideCfg(loadFifoIdReg)
// addrDyn是该流当前tile取数首地址
// strideCfg是该流各Word取数间隔
```

当loadWordCnt达到半区长度时，这一轮的所有访存请求已经全部发送，将访存请求拉低，并根据tileStrideCfg更新addrDyn寄存器

访存请求处理（D1/D2级）

对应流水线中的D1/D2 Stage，逐级流水访存请求、访存地址、写Buffer的地址等信息用于WB级。同样与访存流水线一致，在D1级给L1Cache提供用于mmu的地址（bug）。

```
io.dc.rreqD1      := loadValidRegD1
io.dc.paddrD1     := loadAddrRegD1
```

访存数据写回（WB级）

前面流水级在发送Cache miss堵塞时，需要保持原值，但是对于D2到WB级的更新，当发生Cache miss请求时，该级的所有信号(包括写使能信号)应该清空：

```
val loadWordCntRegWB = WireDefault(ShiftRegister(
  Mux(io.dc.miss || io.dc.sbFull, 0.U, loadWordCntRegD2),
  1,
  0.U((120offset-2).W),
```

```
    true.B  
))
```

写使能、写地址由前面的流水级传递下来，写数据由cache提供，并根据reuseCfg在readyMap中置相应的消费数

Store流仍然直连AXI

V0：流Engine 只连 AXI（原始版本）

Load 流：从主存读数据到 Buffer

粒度：

- 一次从AXI请求一个L2 Cacheline，对应双缓冲结构中的一个半区

条件与限制：

- 为了实现简便，取数一个条件是 Buffer某个半区的所有项 ready = 0（空闲）
- 取数的另一个条件是该流的burst次数未超过其配置次数
- 各流按半区0->半区1->半区0->半区1这样的方式取数，并且半区0取第0、2、4、6...组cacheline数据，半区1取第1、3、5、7...组cacheline数据

过程：

1. **select stream** 从 Buffer 空闲、配置完毕的流中选择一个读主存。
2. **AXI 读请求** 直连 AXI Arbiter，与 L2 竞争带宽。
3. **写入 Buffer：** 依次写入要填入的半区：
4. 写完数据 有如下副作用
 1. **ready = 1**
 2. burstCnt递增
 3. addrCfg递增（当前仅实现线性流）

Store 流：Buffer → 主存

条件：

- store Buffer 某个半区 所有index **ready = 1**

过程：

1. 判断是否可写
2. AXI 写入主存
3. 写回主存有如下副作用
 1. `ready = 0`
 2. `burstCnt`递增
 3. `addrCfg`递增（当前仅实现线性流）

3. CALSTREAM 指令在流水线中的流动 (Dispatch -> Issue -> Execute -> Writeback)

CALSTREAM（流计算指令）会从Buffer中读数，计算，写回，完成一次元素级计算

为了保证正确的数据依赖与吞吐，硬件对 CALSTREAM 做了完整的流水处理：

3.1 Dispatch 阶段：为流计算指令分配 index

① 获取当前这条指令对应的 index

因为 流计算指令在issue级需要index判断是否可以发射，因此需要在issue级之前获取 index，这里选择在dispatch级获取

stream Engine需要根据配置好的规则更新各个流的iCntMap，并为dispatch宽度内的 calstream指令分配正确的iter值。

例如，假设当前icntMap中该流操作数对应的计数器值是31，根据配置可知下一个更新值是0。而dispatch宽度是3，并且第1条和第3条指令都是流计算指令，那么他们的 index值分别应该是 $31 \% 64$ 和 $0 \% 64$ ：

```
index = iterCnt % Buffer_Depth
```

② iCntMap更新

CALSTREAM 在 dispatch 获取index，会对icntMap产生副作用，dispatch级有多少条流计算指令下一周期可通过该级(fire)，icntMap中对应流的计数器就要增加多少，并根据配置判定是否回环：

```
for (b <- 0 until 3) {
  when(PopCount(io.rdIter.fireStreamOp(b)) != 0.U){
    val sum = iCntMap(b) + PopCount(io.rdIter.fireStreamOp(b))
    when(sum < iLimitDyn(b)){
      iCntMap(b) := sum
    }.elsewhen (iRepeatDyn(b) + 1.U === iRepeatCfg(b)){
      iCntMap(b) := sum
      iLimitDyn(b) := iLimitDyn(b) + iLimitCfg(b)
      iRepeatDyn(b) := 0.U
    }.otherwise{
      iCntMap(b) := sum - iLimitCfg(b) //sum - iLimitDyn(b) +
iLimitDyn(b) - iLimitCfg(b)
      iRepeatDyn(b) := iRepeatDyn(b) + 1.U
    }
  }
}
```

可能需要维护分支预测失误下icntMap的恢复

3.2 Issue 阶段：检查 Buffer 是否就绪（指令能不能发射？）

Issue 阶段决定该 CALSTREAM 指令是否可以真正进入执行单元。

对于每个计算指令（例如 $a[i] + b[i] \rightarrow c[i]$ ），Issue 必须检查：

- Load 流的 Buffer 对应 index 的 ready > 0（有数据可消费）
- 若需要写流Buffer，检查 Store 流的 Buffer 对应 index 的 ready == 0（有空位可写）

示例（向量加）：

流	类型	条件
A	Load	readyMap(0)[i] > 0
B	Load	readyMap(1)[i] > 0
C	Store	readyMap(2)[i] == 0

只有三个条件都满足，这条指令才允许发射。

3.3 Execute 阶段：读取 → 计算 → 写回

Execute 包含三个流水级，与普通整数指令风格保持一致：

3.3.1 ReadOp（操作数读取）

流计算指令有特殊的opcode，带有这种opcode的指令会从Buffer 中取出本次计算需要的操作数：

```
rdata1 = Buffer(A)[index]
rdata2 = Buffer(B)[index]
```

3.3.2 ALU 计算

当前版本的 ops 写死为加法(不写通用寄存器) 或 乘法(写通用寄存器)：

未来可以扩展为更多流式计算（乘、MAC、激活函数等），这个是很容易实现的

3.3.3 Writeback（写入 store Buffer / 通用寄存器）

若向 store Buffer 中写入，需要同时更改对应index的flag：

```
Buffer(C)[index] = result
readyMap(C)[index] = 1           // 标记此项可被写回主存
readyMap(A)[index]--             // load 流项已被消费
readyMap(B)[index]--
```

这一步完成后：

- load 流消费数减少，归0时可被下一轮访存填充
- store 流对应 index 变为 ready，可被 AXI 写回主存

4. 向量加（Vector Add） 执行示例

该示例是基于之前的版本讲述，可能与目前版本实现稍有不符

下面以三个流（a、b、c）说明 512 elements vector add：

```
c[i] = a[i] + b[i]
```

4.1 Buffer 分配

每个流分配：

- 一个pingpong buffer（两个半区）：**line0** 和 **line1**
- 每个半区含一个l2 cacheline（32 个 word）
- 每 word 搭配 1bit ready 标志

双缓冲结构保证计算与访存可完全重叠。

4.2 执行步骤

(a) 指令配置

配置：

- a, b 的起始地址
- 每次取 32 word
- 共需 512 word → 16 次 burst

(b) 第一批取数：填 line0

AXI 依次取：

```
a[0..31] → a.line0
```

```
b[0..31] → b.line0
```

ready 置 1。

(c) 第一轮计算（0–31）+ 第二批取数（32–63）

计算单元开始：

```
c[i] = a[i] + b[i], i = 0..31
```

算好后load流相应line0中的ready位会拉低，store流的会拉高

同时 AXI 获取下一批：

```
a[32..63] → line1  
b[32..63] → line1
```

(d) 第二轮计算（32–63）+ 第三批取数（64–95）

第二轮计算开始：

```
c[i] = a[i] + b[i], i = 32..63
```

准备从主存获取下一批（64–95）到 line0。

由于line1已经取好，并且计算速度(line0的消费速度)快于访存(line1的取数速度)，因此两个load流的line0肯定已经被拉低，所以可以取数

```
a[32..63] → line0  
b[32..63] → line0
```

(e) 第 n 轮循环

双缓冲循环往复：

计算本批 (i)
访存下一批 (i+1)

直到 16 轮完成所有 512 word 计算。

4.3 执行效果

- AXI 访存几乎无空闲
- 计算完全被访存隐藏
- 每轮延迟 $\approx$ 两次 32-word 读的耗时
- 增大 burst size 可进一步减少握手带宽损耗

最终实际测量：

$\approx$ 1700-1800 cycle 完成整段 vector add

5 矩阵乘的流

上述向量加的例子中，数据不存在复用（每个元素取回后只访问1次），也不存在复杂的访存pattern（顺序取数）但在矩阵乘中会存在，下面首先简单描述下矩阵内积乘算法 (MNK)：

- 计算C11：由A的**第一行**和B的第一列计算得到
- 计算C12：由A的**第一行**和B的第二列得到
- ...
- 计算C21：由A的**第二行**和B的第一列得到
- 计算C22：由A的**第一行**和B的第二列得到
- ...

我们的设计是，将A的一行和B的一列取到流buffer中，使用**流乘法指令**和**普通加法指令**进行乘累加操作，C的结果写回通用寄存器

为了实现上述算法，需要实现：

1. 流 Engine 按列取数据：数据按列组织在流Buffer中，以便流计算指令能够高效索引数据。
2. 流 Engine 与 Cache 的接口：使用Cache作为流Buffer的后备缓存，节省内存带宽
3. 流 Buffer 的复用：对于某些流，不能像原来一样使用一次之后就无效Buffer数据
4. 特殊的 icntMap 更新逻辑：原有的icntMap只需要递增，现在需要一些回环-重复的模式

5.1 矩阵A内存取数：Stream Buffer复用

我们采用最常见的内积算法，矩阵A的一行被读入到Stream buffer后，各 element 应该被消耗掉N次，这一功能需要有新增的指令来配置：

```
static int cfg_reuse(uint32_t reuse,int fifo_id)
{
    asm volatile (
        ".insn r 0x0b, 4, 0, x0, %0, %1"
        :
        : "r"(reuse), "r"(fifo_id)
    );
    return 0;
}
```

该指令配置数据被读入到Stream Buffer中后被使用的次数，因此数据读入到Stream Buffer时，readymap被初始化为配置的次数：

```
readyMap(A)[index] = reuseCfg(A)
```

每当流计算指令消耗掉一次index位置的数据：

```
readyMap(A)[index] = readyMap(A)[index] - 1
```

直到为0，表示该数据不会再被复用，Stream Buffer中的位置空闲，流Enigne 控制逻辑可以为该Buffer发起向主存的一次新请求

5.2 矩阵B内存取数（Cache复用）

与矩阵A的复用不同，矩阵B的一列被读入Stream Buffer后，暂时只使用一次，与矩阵A的一行进行点积之后，便需要读入矩阵B的**下一列**。当矩阵A的一行消耗完之后，B之前**读过的列**又需要被读入Stream Buffer。

一方面，主存按行burst数据，因此取B的第一列数据时，后续X列（X=burst长度）实际上也会被访问到，而实际上后续X列的数据也确实会在第一列数据使用完后被使用，因此应该把它们缓存到Cache中。

另一方面，当A的一行和B的N列计算结束得到C的一行之后，需要计算C新的一行，这时候B此前访问过的列，会再次依次被访问，因此缓存在Cache中是有用的。

新增了如下设计支持上述功能：

1. **cfg-stride**指令：用来配置某个流 **某次** 向存储系统取数填充 流Buffer 时各个 element的地址间隔。对于矩阵乘示例B数组，该值即为 $N*4$ (Byte)
2. **cfg-tileStride**指令：用来配置某个流 向存储系统取数填充 流Buffer 时，**各次**取数首地址之间的地址间隔。对于矩阵乘A数组，该值即为两行的行首地址间隔 ($K*4$)；对于B，该值为两列的列首地址间隔(4)
3. **cfg-outerIter**指令：该指令搭配**cfg-length**指令使用，当某个流burst的次数达到配置值时，若**outerIter**次数达到配置值，则该流不能再取数；否则取数首地址**回环**到最开始配置的地址，**outerIter**次数++。
4. 考虑利用Cache进行复用，Stream Buffer需要数据时直接去Cache中查找数据是否已经在Cache中，如果命中则直接由Cache提供数据；如果miss，复用Cache向主存访问数据的通路。数据取回后，将一整个Cacheline放到Cache，将流Engine需要的数据给流Engine

5.3 矩阵A B 流计算指令消费：icntMap更新逻辑

提要：

1. 每个流都对应一个**icntMap**，本质上就是一个计数器
2. 一条流计算指令会需要至少2个流操作数（目的操作数可以是通用寄存器）

流计算指令的指令码本身不携带 **各个流操作数 在相应流Buffer中的索引**，各操作数的索引是指令来到dispatch级时 **各流操作数对应的icntMap** 的值。在向量加算法（A+B）

中，dispatch级每fire一条流计算指令，流A和流B 相应的icntMap就会递增1，但是在矩阵乘中不是这样的。

在内积算法中，A的某一行会依次和B的N列进行点积，得到N个C的result；对于每个result，它是由A的这一行的所有element和B的相应列的所有element乘累加得到的。

这里假设流Buffer一个半区是32字（并且K=32），以C的前两行计算为例，它需要A的流Buffer的两个半区依次取好A的前两行各32个数据，并且B的各列按顺序交叠的load到两个半区中。

对于B，它的icntMap更新策略很简单，就是每fire一条流计算指令递增1；

对于A，它的icntMap更新策略是：fire一条流计算指令时，更新值与当前icntMap的值相关：从0递增到31，然后重新回到0再递增到31，重复N次。

具体的过程是这样的：



各条流乘法指令被分配的索引应该是这样的：

第 X 条	A 索引	B 索引	备注
0	0	0	A第1行与B第1列开始计算
1	1	1	
...	...	...	
31	31	31	
32	0	32	A第1行与B第2列开始计算
33	1	33	
...	...	...	
63	31	63	
64	0	64	A第1行与B第3列开始计算
...	...	...	
N×32-1	31	N×32-1	
N×32	32	N×32	A第2行与B第1列开始计算

需要实现2条配置指令，一条配置*i_limit*，即icntMap递增多少时会回环；一条配置*i_repeat*，即icntMap回环多少次后回环累计清0，本次不回环

5.4 流计算指令：新增可写通用寄存器的计算指令

此前在向量加程序中，两个流计算的结果不会作为后续计算的源操作数，因此非常适合把结果存放在流Buffer中，集中写回主存

但在矩阵乘中，对于C的每个result，它是由A某一行所有element和B相应列的所有element乘累加得到的。也就是说，流乘法指令的结果会进行复用。

考虑到通用寄存器本身就具有“复用”的特性，因此新增可写通用寄存器的流计算指令，通过funct3与原指令区分，这条指令的内联汇编如下：

```
static int cal_stream_rd(int fifo_id_src0,int fifo_id_src1)
{
    int rd;
    int rs1 = (fifo_id_src0 & 0x3) | ((fifo_id_src1 & 0x3) << 2);
    asm volatile (
        ".insn r 0x0b, 7, 0, %0, %1, x0"
        : "=r"(rd)
        : "r"(rs1)
        );
    return rd;
}
```

矩阵乘内积算子实现如下：

```
for (int i = 0; i < M*N; i++){
    int sum = 0;
    for (int k = 0; k < K;k++){
        sum += cal_stream_rd(0,1);
    }
    c[i] = sum;
}
```

5.5 Bypass（未实现）

目前允许流直连 AXI 或 直连L2

Bypass指的是L2miss时，通过一些动态机制决定AXI返回的数据要不要存放在L2里

实现Bypass带来的收益是：

1. 在L2-miss的情况下，可以稍微早一些拿到数据（关键字优先，30-40cycles左右）
2. 防止污染L2

实现的复杂度过高，并且收益未可知